

Agile Backlog Delivery Plan: CarCare Web API

Epic 1: Authentication & Identity Management

Epic Description: Provision secure, role-based access control (RBAC), user registration, identity claims management, and JWT-based authentication for the CarCare platform.

Story 1.1: Customer Public Registration

- **User Story:** As a new Customer, I want to register an account with my contact information, so that I can access the system and manage my vehicle appointments.
- **Priority:** High
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** System accepts name, email, phone, and password via POST /api/auth/register.
 - **AC-02:** Public registration strictly provisions Customer accounts only (BR-13). Staff and Manager roles cannot be selected.
 - **AC-03:** Password must meet Identity strength requirements; invalid requests return 400 Bad Request (RFC 7807 ProblemDetails).
 - **AC-04:** Registering an existing email address returns 409 Conflict.
 - **AC-05:** Successfully registered user creates linked ApplicationUser and Customer entity records, returning 201 Created.
- **Subtasks:**
 1. Create ApplicationUser (ASP.NET Core Identity) and Customer EF Core domain entities with foreign key mapping.
 2. Create unique database index on ApplicationUser.Email.
 3. Implement POST /api/auth/register endpoint with FluentValidation for request model.
 4. Implement CustomerService.RegisterAsync to atomically persist Identity user and Customer profile inside a DB transaction.

Story 1.2: User Authentication & JWT Issuance

- **User Story:** As an existing user (Customer, Staff, Manager), I want to log in using my credentials, so that I can obtain a secure token to perform authorized actions.
- **Priority:** Highest
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** POST /api/auth/login validates credentials against stored ASP.NET Core Identity hashes.
 - **AC-02:** Invalid credentials return 401 Unauthorized.
 - **AC-03:** Successful login returns 200 OK with a signed JWT Bearer token containing sub (UserId), email, and role claims, along with expiration details.
- **Subtasks:**
 1. Configure JWT Authentication middleware in ASP.NET Core pipeline (Program.cs).
 2. Create Token Service interface and implementation to generate signed JWTs with custom claims (sub, email, role).
 3. Implement POST /api/auth/login endpoint and response payload DTO.

Story 1.3: Authenticated Profile Retrieval

- **User Story:** As an authenticated user, I want to retrieve my profile information using my token, so that I can verify my identity and role within the application.
- **Priority:** Medium
- **Story Points:** 1
- **Acceptance Criteria:**
 - **AC-01:** GET /api/auth/me requires valid JWT token (BR-01); missing or invalid tokens return 401 Unauthorized.
 - **AC-02:** User identity and claims are extracted directly from the JWT sub claim without relying on query/body parameters (BR-12).
 - **AC-03:** Returns 200 OK with user identity details and linked domain profile information.
- **Subtasks:**
 1. Implement Claims Principal extension method to safely extract UserId from HttpContext.
 2. Implement GET /api/auth/me endpoint with [Authorize] attribute.
 3. Map application user profile data to output DTO.

Epic 2: Vehicle Fleet Management

Epic Description: Enable customers to manage their vehicle records and view service history, while guaranteeing strict data ownership and isolation.

Story 2.1: Register Customer Vehicle

- **User Story:** As a Customer, I want to register my vehicle details in the platform, so that I can schedule service appointments for it.
- **Priority:** High
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** Accepts plateNumber, vin, make, model, and year via POST /api/vehicles.
 - **AC-02:** Year must be within valid range $[\text{1950}, \text{CurrentYear} + 1]$; invalid payload returns 400 Bad Request.
 - **AC-03:** Vehicle is automatically linked to the authenticated customer via server-side JWT claims (BR-02, BR-12).
 - **AC-04:** Duplicate VIN registration returns 409 Conflict. Returns 201 Created on success.
- **Subtasks:**
 1. Create Vehicle EF Core entity with foreign key constraint to Customer(Id).
 2. Add unique database index on Vehicle.Vin and lookup index on Vehicle.CustomerId.
 3. Add FluentValidation rules for VIN length (17), Plate format, and Year bounds.
 4. Implement POST /api/vehicles endpoint with claims-based customer ID mapping.

Story 2.2: View Customer's Owned Vehicles

- **User Story:** As a Customer, I want to view a list of all vehicles I have registered, so that I can select a vehicle when booking a service.
- **Priority:** Medium
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** GET /api/vehicles/my requires Customer role authorization.
 - **AC-02:** Filters vehicles strictly by the authenticated customer's ID derived from JWT (BR-02, BR-12).

- **AC-03:** Returns 200 OK with an array of owned vehicles.
- **Subtasks:**
 1. Implement GET /api/vehicles/my endpoint in VehiclesController.
 2. Implement service query filtering Vehicle entity set by authenticated CustomerId.

Story 2.3: View Vehicle Details with Ownership Enforcement

- **User Story:** As an authorized user, I want to view specific details of a vehicle, so that I can review its specifications and ownership context.
- **Priority:** Medium
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** GET /api/vehicles/{id} returns 200 OK with vehicle details if the caller is Staff/Manager or the owning Customer.
 - **AC-02:** Returns 403 Forbidden if a Customer attempts to query a vehicle owned by another user (BR-02).
 - **AC-03:** Returns 404 Not Found if the vehicle ID does not exist.
- **Subtasks:**
 1. Implement GET /api/vehicles/{id} endpoint.
 2. Add authorization checks validating caller's role or ownership constraint (Vehicle.CustomerId == CurrentUser.CustomerId).

Story 2.4: View Vehicle Service History

- **User Story:** As a Customer or Staff member, I want to view the complete past appointment and service log for a vehicle, so that I can understand its repair history.
- **Priority:** Medium
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** GET /api/vehicles/{id}/service-history returns all past appointments and service snapshot logs.
 - **AC-02:** Access restricted to Staff, Manager, or the owning Customer (BR-02). Attempts by non-owners return 403 Forbidden.
- **Subtasks:**
 1. Create DB query joining Vehicle, Appointment, and AppointmentService with filtering for completed/past bookings.
 2. Implement GET /api/vehicles/{id}/service-history endpoint with authorization filter.

Epic 3: Service Catalog Administration

Epic Description: Provide administrative capabilities to manage center services, catalog pricing, and operational status without corrupting historical records.

Story 3.1: View Active Service Catalog

- **User Story:** As a Customer or Guest, I want to view active maintenance services, so that I can see available options, estimated durations, and pricing.
- **Priority:** High
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** GET /api/services is publicly accessible or authenticated (BR-01).

- **AC-02:** Returns 200 OK listing services where IsActive == true (BR-03).
- **Subtasks:**
 1. Create Service EF Core entity (Id, Name, Description, DurationMinutes, BasePrice, IsActive, CreatedAt).
 2. Implement GET /api/services endpoint returning filtered catalog entries.

Story 3.2: Create New Service Offering

- **User Story:** As a Manager, I want to add new service offerings to the catalog, so that customers can book new types of vehicle maintenance.
- **Priority:** Medium
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** POST /api/services accepts service details (Name, Description, DurationMinutes, BasePrice).
 - **AC-02:** Restricted to Manager role; non-managers receive 403 Forbidden.
 - **AC-03:** Duration must be > 0 and BasePrice ≥ 0 ; invalid inputs return 400 Bad Request. Returns 201 Created on success.
- **Subtasks:**
 1. Add FluentValidation rules for DurationMinutes and BasePrice.
 2. Implement POST /api/services endpoint decorated with [Authorize(Roles = "Manager")].

Story 3.3: Update Catalog Service & Price Snapshot Safeguard

- **User Story:** As a Manager, I want to update service pricing and details in the catalog, so that prices reflect current center costs without altering historical booking records.
- **Priority:** Medium
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** PUT /api/services/{id} updates Service entity attributes (BasePrice, DurationMinutes, Name, Description).
 - **AC-02:** Restricted strictly to Manager role.
 - **AC-03:** Updating catalog BasePrice does not alter existing AppointmentService.UnitPrice snapshot values on existing appointments (BR-10).
- **Subtasks:**
 1. Implement PUT /api/services/{id} endpoint with Manager authorization.
 2. Write unit/integration test verifying historical AppointmentService.UnitPrice remains unchanged after Service.BasePrice update.

Story 3.4: Service Deactivation

- **User Story:** As a Manager, I want to deactivate a service, so that it cannot be selected for new bookings while preserving historical appointment data.
- **Priority:** Medium
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** PATCH /api/services/{id}/deactivate sets IsActive = false.
 - **AC-02:** Restricted to Manager role. Returns 404 Not Found if service ID does not exist.
 - **AC-03:** Deactivated service cannot be selected in new appointment requests (returns 400 Bad Request on booking attempt) (BR-03).

- **AC-04:** Existing historical appointments containing this service remain valid and untouched (BR-11).
- **Subtasks:**
 1. Implement PATCH /api/services/{id}/deactivate endpoint.
 2. Add validation in appointment booking logic checking IsActive flag.

Epic 4: Appointment Lifecycle & Conflict-Free Scheduling

Epic Description: Core scheduling engine managing multi-service booking, mathematical overlap conflict checks, price snapshotting, atomic state transactions, and strict status transition matrices.

Story 4.1: Book Multi-Service Appointment with Atomic Overlap Check

- **User Story:** As a Customer, I want to book a multi-service appointment for my vehicle at a future time, so that I can get my car serviced without scheduling collisions.
- **Priority:** Highest
- **Story Points:** 8
- **Acceptance Criteria:**
 - **AC-01:** Accepts vehicleId, serviceIds, appointmentDate (UTC), and notes via POST /api/appointments.
 - **AC-02:** Validates appointmentDate > DateTime.UtcNow (BR-04) and all selected services have IsActive == true (BR-03); fails with 400 Bad Request otherwise.
 - **AC-03:** Validates vehicle ownership against authenticated user claims (BR-02); fails with 403 Forbidden if unowned.
 - **AC-04:** Computes total duration: $\text{TotalDuration} = \sum \text{Service.DurationMinutes}$.
 - **AC-05:** Calculates overlap window [Start,End] and checks active bookings (Status\in Requested, Confirmed, InService). Rejects booking with 409 Conflict if ExistingStart < NewEnd) (ExistingEnd > NewStart) (BR-05).
 - **AC-06:** Executes overlap check and persistence inside an atomic database transaction with pessimistic/optimistic concurrency locking (BR-14).
 - **AC-07:** Snapshots current Service.BasePrice into AppointmentService.UnitPrice (BR-10).
 - **AC-08:** Enqueues asynchronous background notification job for staff and returns 201 Created.
- **Subtasks:**
 1. Create Appointment and AppointmentService EF Core entities with composite keys and composite index (VehicleId, Status, AppointmentDate).
 2. Build overlap calculation engine applying explicit mathematical formula in EF Core query.
 3. Wrap creation logic in EF Core Execution Strategy / DB Transaction (IDbContextTransaction) with row-level locking mechanism.
 4. Implement POST /api/appointments endpoint with full validation pipeline.

Story 4.2: View Customer's Appointments

- **User Story:** As a Customer, I want to view my appointment history and current statuses,

so that I can stay updated on my service requests.

- **Priority:** High
- **Story Points:** 2
- **Acceptance Criteria:**
 - **AC-01:** GET /api/appointments/my returns caller's appointments derived from JWT claims (BR-12).
 - **AC-02:** Returns 200 OK with array containing status, scheduled date, snapshot prices, and service breakdowns.
- **Subtasks:**
 1. Implement GET /api/appointments/my endpoint.
 2. Implement LINQ query projecting Appointment and AppointmentService entities into DTOs.

Story 4.3: Customer Self-Cancellation

- **User Story:** As a Customer, I want to cancel my pending appointment, so that I can free up space if my plans change.
- **Priority:** High
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** DELETE /api/appointments/{id} allows cancellation if caller owns the appointment (BR-06). Unowned requests return 403 Forbidden.
 - **AC-02:** Allowed only if status is Requested or Confirmed. Sets status to Cancelled and updates CancelledAt = DateTime.UtcNow.
 - **AC-03:** If status is InService or Completed, request is rejected under BR-07 with 400 Bad Request (RFC 7807 payload).
- **Subtasks:**
 1. Implement DELETE /api/appointments/{id} endpoint.
 2. Add status validation guard checking for lock state (InService, Completed).
 3. Update appointment record and persist CancelledAt timestamp.

Story 4.4: Staff State Machine Transition Management

- **User Story:** As a Staff/Manager, I want to update an appointment's operational status, so that the center workflow moves accurately through service stages.
- **Priority:** Highest
- **Story Points:** 5
- **Acceptance Criteria:**
 - **AC-01:** PUT /api/appointments/{id}/status requires Staff or Manager role (BR-09); non-staff callers receive 403 Forbidden.
 - **AC-02:** Enforces forward-only State Machine Matrix (BR-08):
 - Requested ---> Confirmed | Cancelled
 - Confirmed ---> InService | Cancelled
 - InService ---> Completed | NoShow
 - Terminal states (Completed, Cancelled, NoShow) accept no further changes.
 - **AC-03:** Invalid transition attempts return 400 Bad Request with problem details explaining valid state paths.
 - **AC-04:** Successful transition updates UpdatedAt timestamp, returns 200 OK, and triggers

asynchronous customer notification.

- **Subtasks:**

1. Implement State Machine validator class enforcing allowed state transitions.
2. Implement PUT /api/appointments/{id}/status with [Authorize(Roles = "Staff,Manager")].
3. Enqueue asynchronous customer status change notification upon DB commit.

Epic 5: Automated Operations & Background Processing

Epic Description: Offload recurring maintenance tasks, stale appointment cleanups, and asynchronous notifications to background processing channels.

Story 5.1: Automatic Stale Appointment Cleanup Job

- **User Story:** As a System Administrator, I want the system to automatically cancel stale unconfirmed appointments, so that garage calendar slots are not blocked indefinitely.
- **Priority:** High
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** Hangfire background job runs on a recurring hourly schedule (BR-15).
 - **AC-02:** Scans for appointments with Status == Requested and CreatedAt < DateTime.UtcNow.AddHours(-24).
 - **AC-03:** Automatically updates identified records to Status = Cancelled and CancelledAt = DateTime.UtcNow.
 - **AC-04:** Job executes reliably in background without interfering with live HTTP transactions.
- **Subtasks:**
 1. Integrate and configure Hangfire framework with SQL Server storage in ASP.NET Core.
 2. Create IStaleAppointmentCleanupJob service and implementation.
 3. Register recurring hourly cron job (RecurringJob.AddOrUpdate).

Story 5.2: Asynchronous Notification Processing

- **User Story:** As a User, I want to receive timely notifications upon appointment updates, so that I am kept informed without slowing down API performance.
- **Priority:** Medium
- **Story Points:** 3
- **Acceptance Criteria:**
 - **AC-01:** Appointment creation enqueues background job notifying Staff.
 - **AC-02:** Status transitions enqueue background job notifying the Customer.
 - **AC-03:** Background notification failures log errors without rolling back primary API database transactions.
- **Subtasks:**
 1. Define notification contracts and Hangfire background job handlers (INotificationJobService).
 2. Integrate notification enqueue calls inside Appointment creation and status transition workflows.
 3. Configure logging and retry policies for background job failures.

Backlog Summary Matrix

Epic	Story ID	Title	Priority	Story Points
Epic 1: Authentication & Identity	1.1	Customer Public Registration	High	3
	1.2	User Authentication & JWT Issuance	Highest	3
	1.3	Authenticated Profile Retrieval	Medium	1
Epic 2: Vehicle Fleet Management	2.1	Register Customer Vehicle	High	3
	2.2	View Customer's Owned Vehicles	Medium	2
	2.3	View Vehicle Details with Ownership Check	Medium	2
	2.4	View Vehicle Service History	Medium	3
Epic 3: Service Catalog Administration	3.1	View Active Service Catalog	High	2

n				
	3.2	Create New Service Offering	Medium	2
	3.3	Update Catalog Service & Price Safeguard	Medium	2
	3.4	Service Deactivation	Medium	2
Epic 4: Appointment Lifecycle	4.1	Book Multi-Service Appointment (Atomic Overlap)	Highest	8
	4.2	View Customer's Appointments	High	2
	4.3	Customer Self-Cancellation	High	3
	4.4	Staff State Machine Transition Management	Highest	5
Epic 5: Automated Operations	5.1	Automatic Stale Appointment Cleanup Job	High	3

	5.2	Asynchronous Notification Processing	Medium	3
Total Story Points				51